

Drafting for CA096: Declaration matching for non-dependent requires-clauses

Document Number: P1980R0

Date: 2019-11-08

Audience: Core Working Group

Author: Jason Merrill

National Body Comment CA096: Declaration matching for non-dependent requires-clauses

Declaration matching ([over.dcl]) is based upon whether trailing requires-clauses are equivalent; however, equivalent, with respect to expressions ([temp.over.link]), is defined only for expressions involving template parameters.

Proposed change:

Extend the definitions of equivalent and functionally equivalent to cover expressions subject to normalization in general (not just those involving template parameters).

Further, make the determination of expression equivalence treat concept definitions as opaque by adding a condition that an expression that may be subject to constraint normalization is functionally equivalent only if each qualified-concept-name that may be expanded by normalization would be considered to name the same type if, instead of a concept, a class template was named.

Drafting note

This drafting clarifies the status of non-dependent requires-clauses by pointing out that they are unevaluated operands, and therefore contained concept-ids are not evaluated unless they appear in a manifestly constant-evaluated expression.

Importantly, a constraint-expression is no longer manifestly constant-evaluated, only the result of atomic constraint expression substitution during satisfaction.

This also resolves US095.

Proposed Resolution

Remove 7.5.4 [expr.prim.id] (to move it to clause 13):

An id-expression that denotes the specialization of a concept (13.7.8) results in a prvalue of type bool. The expression is true if the concept's normalized constraint-expression (13.5.2) is satisfied (13.5.1) by the specified template arguments and false otherwise. [Example:

```
template<typename T> concept C = true;  
static_assert(C<int>); // OK
```

--end example] [Note: A concept's constraints are also considered when using a template name (13.3) and during overload resolution (Clause 12), and they are compared during the partial ordering of constraints (13.5.4). --end note]

Change 7.7 [expr.const]/12-13:

An expression or conversion is in an immediate function context if it is potentially evaluated and its innermost non-block scope is a function parameter scope of an immediate function. An expression or conversion is an immediate invocation if it is an explicit or implicit invocation of an immediate function and is not in an immediate function context. An immediate invocation shall be a constant expression. ~~[Note: An immediate invocation is evaluated even in an unevaluated operand. --end note]~~

An expression or conversion *e* is manifestly constant-evaluated if it is:

- ...
- a constraint-expression (13.5.2) including one formed from the constraint-logical-or-expression of a requires-clause the result of substitution into an atomic constraint expression to determine whether it is satisfied (13.5.1.2), or
- ...

[Note: A manifestly constant-evaluated expression is evaluated even in an unevaluated operand. --end note]

Change 12.3 [over.dcl]:

Two function declarations of the same name refer to the same function if they are in the same scope and have equivalent parameter declarations (12.2) and equivalent (13.7.6.1) trailing requires-clauses, if any (9.3). [Note: since a constraint-expression is an unevaluated operand, equivalence compares the expressions without evaluating them. [Example:

```
template<int I> concept C = true;
template<typename T> struct A {
    void f() requires C<42>; // #1
    void f() requires true; // OK, different functions
};
```

--end example] --end note]

A function member of a derived class is not in the same scope as a function member of the same name in a base class.

No change to 13.1 [temp.pre]/10:

A template-declaration is written in terms of its template parameters. The optional requires-clause following a template-parameter-list allows the specification of constraints (13.5.2) on template arguments (13.4). The requires-clause introduces the constraint-expression that results from interpreting the constraint-logical-or-expression as a constraint-expression. The constraint-logical-or-expression of a requires-clause is an unevaluated operand (Clause 7). [Note: The expression in a requires-clause uses a restricted grammar to avoid ambiguities. Parentheses can be used to specify arbitrary expressions in a requires-clause. [Example:

```
template<int N> requires N == sizeof new unsigned short
int f(); // error: parentheses required around == expression
```

-- end example] -- end note]

Add to the end of 13.3 [temp.names]:

A concept-id is a simple-template-id where the template-name is a concept-name. A concept-id is a prvalue of type bool, and does not name a template specialization. A concept-id evaluates to true if the concept's normalized constraint-expression (13.5.2) is satisfied (13.5.1) by the specified template

arguments and false otherwise. [Note: Since a constraint-expression is an unevaluated operand, a concept-id appearing in a constraint-expression is not evaluated except as necessary to determine whether the normalized constraints are satisfied. --end note] [Example:

```
template<typename T> concept C = true;  
static_assert(C<int>); // OK
```

-- end example]

Change 13.5.3 [temp.constr.normal]/1.4:

- The normal form of ~~an id-expression of the form~~ a concept-id $C\langle A_1, A_2, \dots, A_n \rangle$, where ~~C names a concept~~, is the normal form of the constraint-expression of C, after substituting $A_1, A_2, \dots, A_n$ for C's respective template parameters in the parameter mappings in each atomic constraint.

Change 13.7.6.1 [temp.over.link]/5:

Two expressions involving template parameters are considered equivalent if two function definitions containing the expressions would satisfy the one-definition rule (6.3), except that the tokens used to name the template parameters may differ as long as a token used to name a template parameter in one expression is replaced by another token that names the same template parameter in the other expression. Two unevaluated operands that do not involve template parameters are considered equivalent if two function definitions containing the expressions would satisfy the one-definition rule, except that the tokens used to name types and declarations may differ as long as they name the same entities, and the tokens used to form concept-ids may differ as long as the two template-ids would be considered to name the same type if, instead of a concept, a class template were named. [Note: For instance, $A\langle 42 \rangle$ and $A\langle 40+2 \rangle$ name the same type. --end note] Two lambda-expressions are never considered equivalent. [Note: The intent is to avoid lambda-expressions appearing in the signature of a function template with external linkage. --end note] For determining whether two dependent names (13.8.2) are equivalent, only the name itself is considered, not the result of name lookup in the context of the template. If multiple declarations of the same function template differ in the result of this name lookup, the result for the first declaration is used. [Example:

....

--end example] Two potentially-evaluated expressions involving template parameters that are not equivalent are functionally equivalent if, for any given set of template arguments, the evaluation of the expression results in the same value. Two unevaluated operands that are not equivalent are functionally equivalent if, for any given set of template arguments, the expressions perform the same operations in the same order with the same entities. [Note: For instance, one could have redundant parentheses. --end note]

Change 13.7.8 [temp.concept]/5:

A concept is not instantiated (13.9). [Note: ~~An id-expression that denotes a concept specialization~~ A concept-id (13.3) is evaluated as an expression (7.5.4) . A concept cannot be explicitly instantiated (13.9.2), explicitly specialized (13.9.3), or partially specialized. -- end note]

The constraint-expression of a concept-definition is an unevaluated operand.